



Programacion Orientada a Objetos **2**

Ing. Jose Rusca
Ing. Federico Stulich



Design Patterns

Clase de hoy



- Definición
- Patrones creacionales: Singleton, Factory.
- Estructurales: Composite, Decorator.
- De Comportamiento: State, Strategy, Observer.

What Is a Design Pattern?



Un patrón describe un problema que se repite constantemente en nuestro entorno. Describe la esencia de la solución a dicho problema, de tal manera que se pueda aplicar esa solución innumerables veces, sin tener que repetirla exactamente igual cada vez.



Creational Patterns

Singleton
Factory

JS

Creational Patterns - Singleton



Asegura que una clase tenga una única instancia y proporciona un punto de acceso global a la misma.

Creational Patterns - Singleton - Aplicación



- Debe haber solo una instancia de una clase, y esta debe ser accesible para los clientes desde un punto de acceso conocido.
- Cuando la única instancia debe ser extensible mediante subclases, y los clientes deben poder usar una instancia extendida sin modificar su código.

Creational Patterns - Singleton - Características



- Acceso controlado a una única instancia.
- Espacio de nombres reducido.
- Permite la optimización de las operaciones y la representación.
- Permite un número variable de instancias.
- Más flexible que las operaciones de clase.

Creational Patterns - Singleton - con Clases

```
class Singleton {  
  constructor() {  
    if (Singleton.instance) {  
      return Singleton.instance;  
    }  
    this.value = Math.random();  
    Singleton.instance = this;  
  }  
  getValue() {  
    return this.value;  
  }  
}
```

```
// Ejemplo de uso:  
const singletonA = new Singleton();  
const singletonB = new Singleton();  
  
console.log(singletonA === singletonB); // true  
console.log(singletonA.getValue());     // Por ejemplo: 0.123456  
console.log(singletonB.getValue());     // El mismo valor que singletonA
```



Creational Patterns - Singleton - con Prototipos

```
const Singleton = (function() {  
  let instance;  
  return function() {  
    if (!instance) instance = {};  
    return instance;  
  };  
})();
```

```
const a = Singleton();  
const b = Singleton();  
console.log(a === b); // true
```



Creational Patterns - Factory



Define una interfaz para la creación de objetos, pero deja que las subclases decidan qué clase se debe instanciar. El patrón de diseño Factory Method permite que una clase delegue la creación de instancias a las subclases.

Creational Patterns - Factory - Aplicación



- Una clase no puede anticipar la clase de objetos que debe crear.
- Una clase quiere que sus subclases especifiquen los objetos que crea.
- Las clases delegan la responsabilidad a una de varias subclases auxiliares, y se desea localizar el conocimiento de qué subclase auxiliar es la delegada.

Creational Patterns - Factory - Características



- Permite la creación de métodos específicos para las subclases.
Crear objetos dentro de una clase mediante un método que siempre resulta más flexible que crear un objeto directamente.
- Facilita la integración de jerarquías de clases paralelas.
En los ejemplos vistos hasta ahora, el *factory method* sólo es invocado por la clase creadora. Sin embargo, no es necesario que sea así; los clientes también pueden beneficiarse de los *factory method*, especialmente en el caso de jerarquías de clases paralelas.

Creational Patterns - Factory - Ejemplos Simple

```
// Constructores de los diferentes tipos
function Admin(name) {
  this.role = 'admin';
  this.name = name;
}
Admin.prototype.sayHello = function() {
  return `Hola, soy ${this.name} y soy administrador.`;
};

function Guest(name) {
  this.role = 'guest';
  this.name = name;
}
Guest.prototype.sayHello = function() {
  return `Hola, soy ${this.name} y soy invitado.`;
};
```



Creational Patterns - Factory

```
function Member(name) {  
  this.role = 'member';  
  this.name = name;  
}  
  
Member.prototype.sayHello = function() {  
  return `Hola, soy ${this.name} y soy miembro.`;  
};  
  
// La función Factory  
function UserFactory(role, name) {  
  switch (role) {  
    case 'admin':  
      return new Admin(name);  
    case 'guest':  
      return new Guest(name);  
    case 'member':  
      return new Member(name);  
    default:  
      throw new Error('Rol no válido');  
  }  
}
```



Creational Patterns - Factory

```
// Uso del patrón Factory

var user1 = UserFactory('admin', 'Ana');

var user2 = UserFactory('guest', 'Luis');

var user3 = UserFactory('member', 'Carlos');

console.log(user1.sayHello()); // Hola, soy Ana y soy administrador.
console.log(user2.sayHello()); // Hola, soy Luis y soy invitado.
console.log(user3.sayHello()); // Hola, soy Carlos y soy miembro.
```





Estructural Patterns

Composite
Decorator



JS

Estructural Patterns - Composite



Organiza los objetos en estructuras de tipo árbol para representar jerarquías y elementos.

Permite que los clientes traten los objetos individuales y las agrupaciones de objetos de forma uniforme.

Estructural Patterns - Composite - Aplicación



- Desea representar jerarquías de objetos que combinan distintos tipos de objetos.
- Desea que los clientes puedan ignorar la diferencia entre composiciones de objetos y objetos individuales. Los clientes tratarán todos los objetos de la estructura compuesta de forma uniforme.

Estructural Patterns - Composite - Características



- Define jerarquías de clases compuestas por objetos primitivos y objetos compuestos.

Los objetos primitivos pueden combinarse para formar objetos más complejos, los cuales, pueden combinarse entre sí, y así sucesivamente. En cualquier lugar donde el código cliente espere un objeto primitivo, también puede recibir un objeto compuesto.

Estructural Patterns - Composite - Características



- Simplifica el código cliente.

Los clientes pueden tratar las estructuras compuestas y los objetos individuales de forma uniforme. Normalmente, el cliente no necesita saber (ni debería preocuparse por) si está trabajando con un objeto simple o un componente compuesto. Esto simplifica el código cliente, ya que evita la necesidad de escribir funciones con estructuras condicionales (como switch-case) para las clases que definen la composición.

Estructural Patterns - Composite - Características



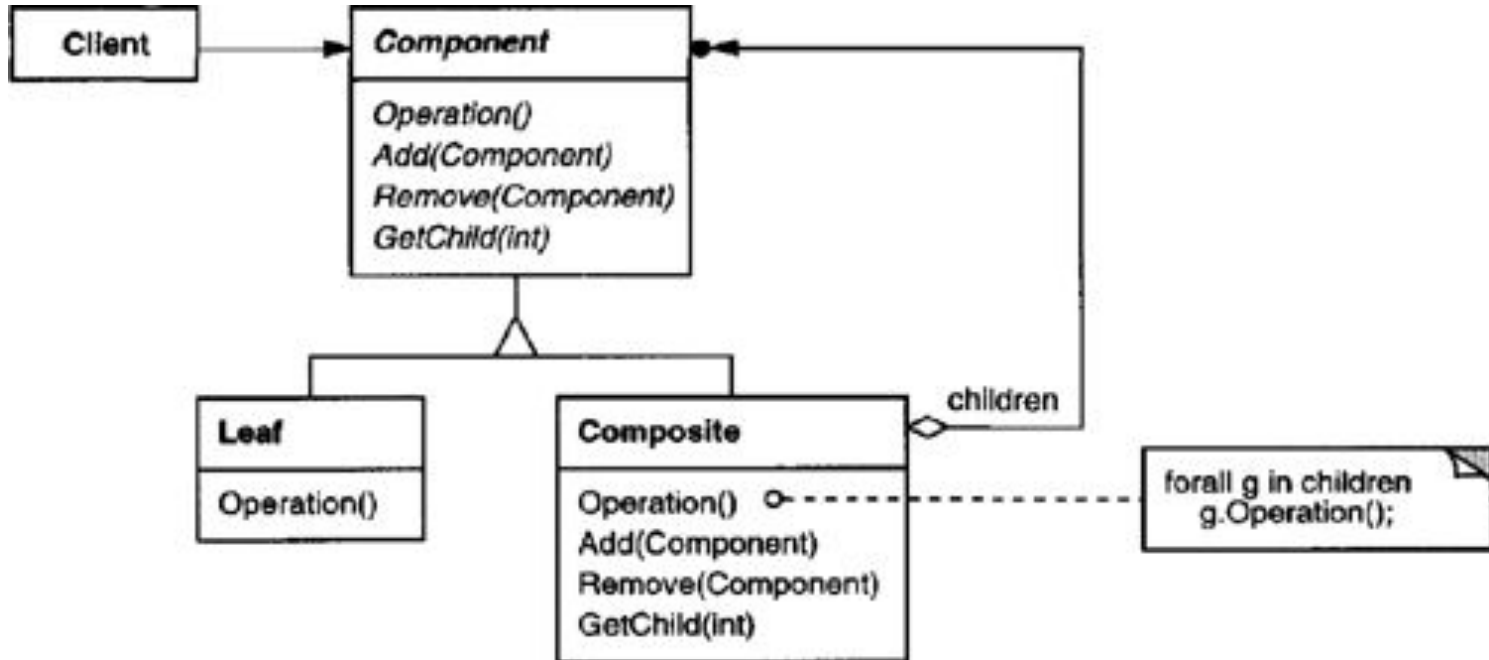
- Facilita la incorporación de nuevos tipos de componentes.
Las nuevas subclases de componentes (tanto compuestos como simples) funcionan automáticamente con las estructuras y el código de cliente existentes. No es necesario modificar el código de cliente para incorporar nuevas clases de componentes.

Estructural Patterns - Composite - Características

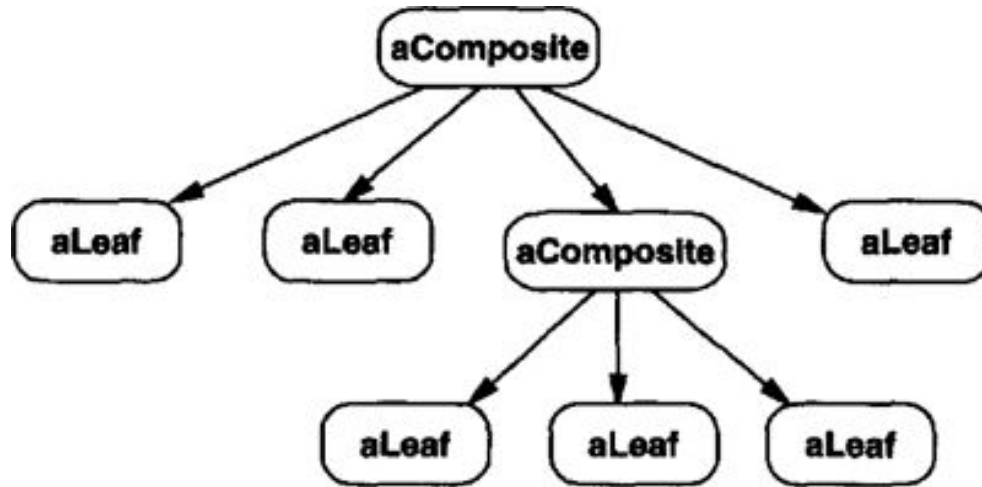


- Puede hacer que el diseño sea demasiado genérico.
La desventaja de facilitar la incorporación de nuevos componentes es que dificulta la restricción de los componentes de un objeto compuesto. A veces, es necesario que un objeto compuesto contenga únicamente ciertos componentes. Con el patrón Composite, no se puede confiar en el sistema de tipos para imponer estas restricciones. En su lugar, será necesario implementar comprobaciones en tiempo de ejecución.

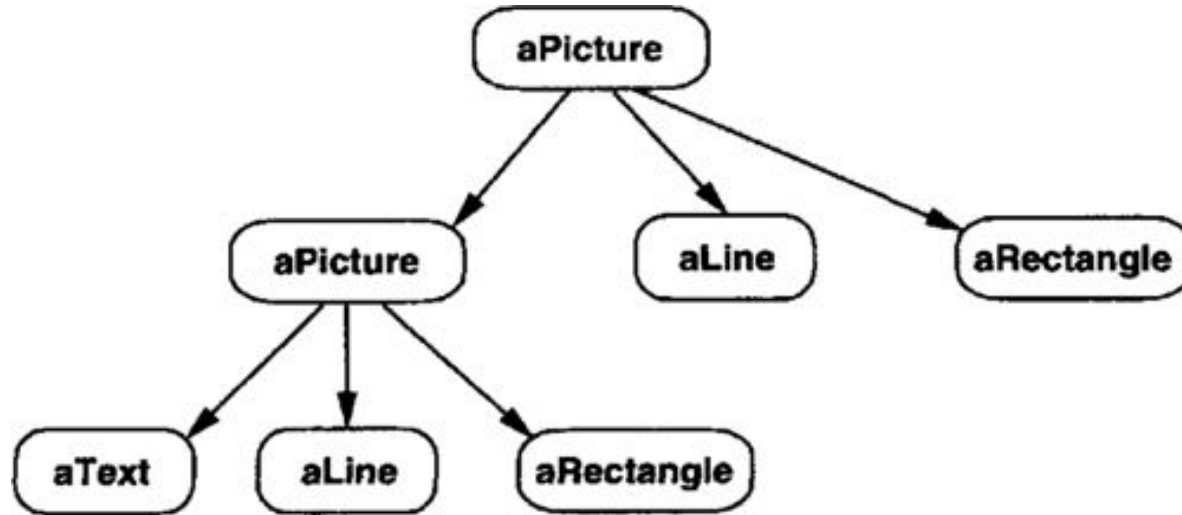
Estructural Patterns - Composite



Estructural Patterns - Composite



Estructural Patterns - Composite



Estructural Patterns - Composite

```
function createLeaf() {  
  return { show: () => console.log("Leaf") };  
}  
  
function createComposite() {  
  const children = [];  
  return {  
    add: c => children.push(c),  
    show: () => {  
      console.log("Composite");  
      children.forEach(child => child.show());  
    }  
  };  
}  
  
const root = createComposite();  
root.add(createLeaf());  
root.add(createLeaf());  
root.show();  
// Composite  
// Leaf  
// Leaf
```



Estructural Patterns - Decorator



Asignar responsabilidades adicionales a un objeto dinámicamente. Los decoradores ofrecen una alternativa flexible a la subclasificación para ampliar la funcionalidad.

Estructural Patterns - Decorator - Aplicación



- Añadir responsabilidades a objetos individuales de forma dinámica y transparente, es decir, sin afectar a otros objetos.
- Eliminar responsabilidades
- Cuando la extensión mediante subclases no es práctica.
A veces, es posible un gran número de extensiones independientes, lo que produciría una proliferación de subclases para todas las combinaciones.

Estructural Patterns - Decorator - Características



- Mayor flexibilidad que la herencia estática.
El patrón Decorator proporciona una forma más flexible de añadir responsabilidades a los objetos que la herencia estática (múltiple).

Con los decoradores, se pueden añadir y eliminar responsabilidades en tiempo de ejecución simplemente asociándolas y desvinculándolas. En cambio, la herencia requiere la creación de una nueva clase para cada responsabilidad adicional.

Estructural Patterns - Decorator - Características



- Evita clases con muchas funciones en los niveles superiores de la jerarquía.

Decorator ofrece un enfoque de pago por uso para añadir responsabilidades. En lugar de intentar incluir todas las funciones previsibles en una clase compleja y personalizable, puedes definir una clase simple y añadir funcionalidad gradualmente con objetos Decorator.

Estructural Patterns - Decorator - Características



- Un decorator y su componente no son idénticos.
Un decorator actúa como un contenedor transparente. Sin embargo, desde el punto de vista de la identidad del objeto, un componente decorado no es idéntico al componente en sí. Por lo tanto, no se debe confiar en la identidad del objeto al usar decorators.

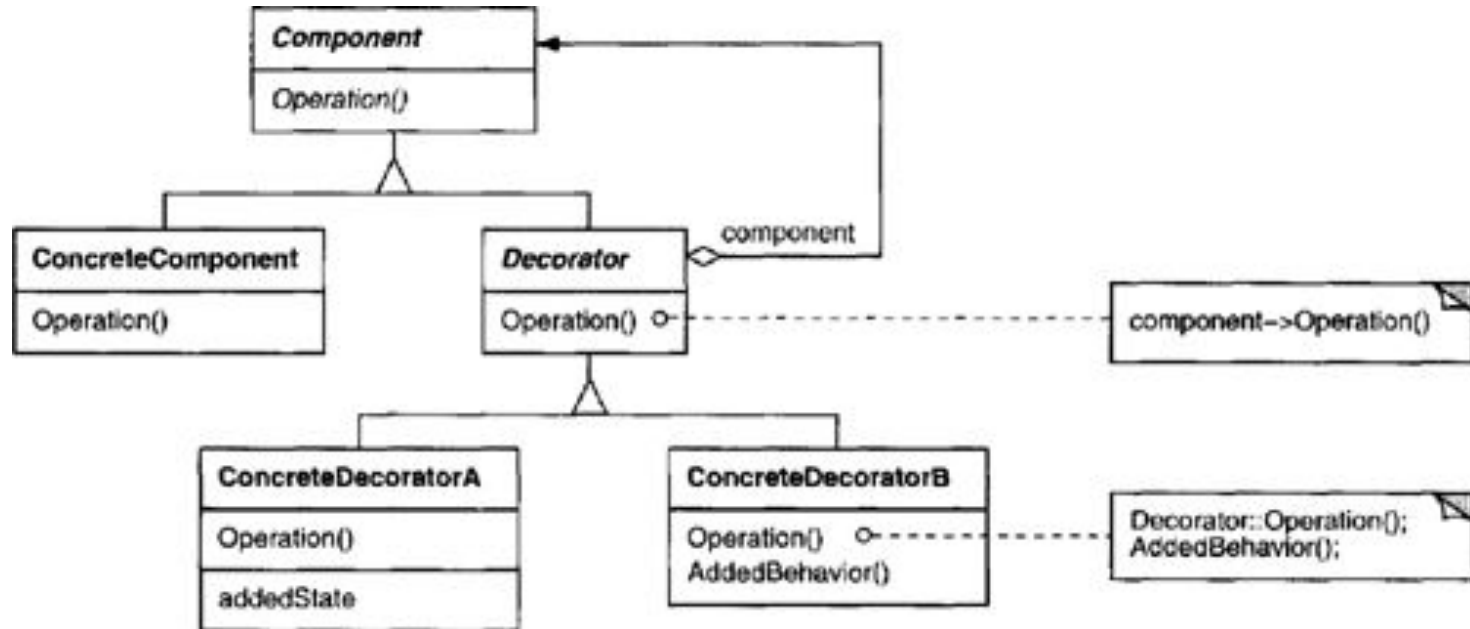
Estructural Patterns - Decorator - Características



- Muchos objetos pequeños.

Un diseño que utiliza Decorator suele generar sistemas compuestos por muchos objetos pequeños que se parecen entre sí. Los objetos difieren únicamente en su interconexión, no en su clase ni en el valor de sus variables. Si bien estos sistemas son fáciles de personalizar para quienes los entienden, pueden ser difíciles de aprender y depurar.

Estructural Patterns - Decorator



Estructural Patterns - Decorator

```
function coffee() {  
  return { cost: () => 5, description: () => 'Café' };  
}  
  
function milkDecorator(coffeeObj) {  
  return {  
    cost: () => coffeeObj.cost() + 2,  
    description: () => coffeeObj.description() + ' + leche'  
  };  
}  
  
function sugarDecorator(coffeeObj) {  
  return {  
    cost: () => coffeeObj.cost() + 1,  
    description: () => coffeeObj.description() + ' + azúcar'  
  };  
}
```



Estructural Patterns - Decorator

```
// Café simple
let c = coffee();
console.log(c.cost(), c.description());           // 5 'Café'

// Café con leche
let milkCoffee = milkDecorator(c);
console.log(milkCoffee.cost(), milkCoffee.description()); // 7 'Café + leche'

// Café con leche y azúcar
let milkSugarCoffee = sugarDecorator(milkDecorator(c));
console.log(milkSugarCoffee.cost(), milkSugarCoffee.description()); // 8 'Café + leche +
azúcar'

// Café solo con azúcar
let sugarCoffee = sugarDecorator(c);
console.log(sugarCoffee.cost(), sugarCoffee.description()); // 6 'Café + azúcar'
```





Behavioral Patterns

State

Strategy

Observer



JS

Behavioral Patterns - State



Permite que un objeto modifique su comportamiento cuando cambia su estado interno. El objeto parecerá cambiar de clase.

Behavioral Patterns - State - Aplicación



El comportamiento de un objeto depende de su estado y debe cambiar su comportamiento en tiempo de ejecución dependiendo de ese estado.

Las operaciones tienen sentencias condicionales extensas y dependen del estado del objeto.

Este estado suele representarse mediante una o más constantes enumeradas. A menudo, varias operaciones contienen la misma estructura condicional.

El patrón "Estado" asigna cada rama de la condicional a una clase independiente.

Behavioral Patterns - State - Características



- Localiza el comportamiento específico de cada estado y lo divide en particiones para diferentes estados.

El patrón Estado integra todo el comportamiento asociado a un estado específico en un solo objeto. Dado que todo el código específico de cada estado reside en una subclase Estado, se pueden añadir fácilmente nuevos estados y transiciones definiendo nuevas subclases.

Behavioral Patterns - State - Características



- Hace explícitas las transiciones de estado.
Cuando un objeto define su estado actual únicamente en términos de valores de datos internos, sus transiciones de estado no tienen una representación explícita; solo se muestran como asignaciones a algunas variables.

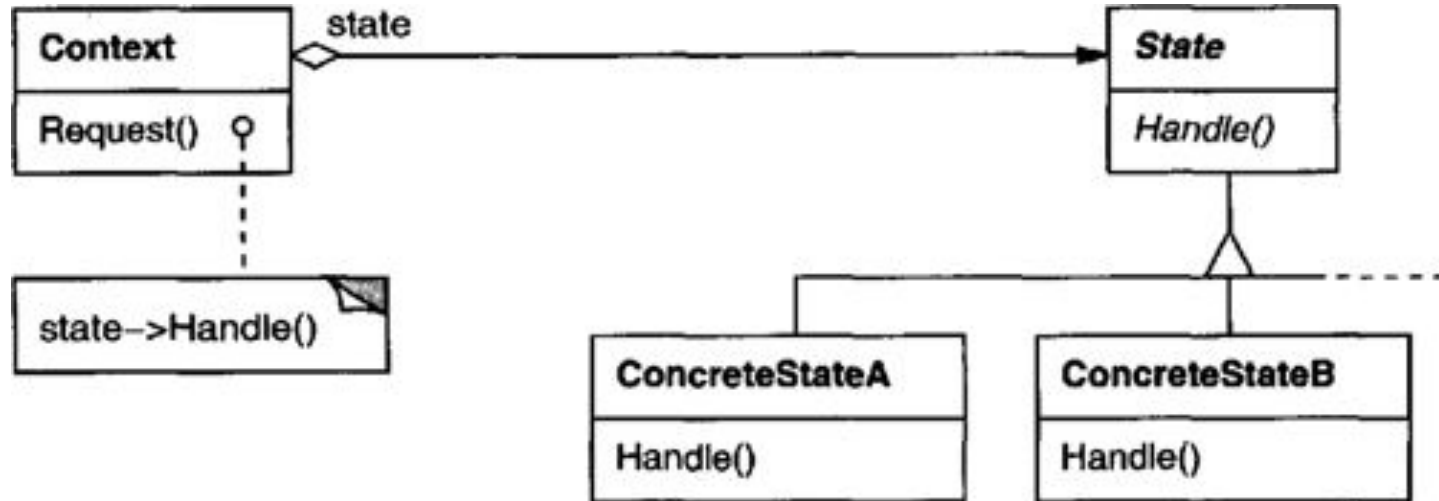
Introducir objetos separados para diferentes estados hace que las transiciones sean más explícitas.

Behavioral Patterns - State - Características



- Los objetos de estado se pueden compartir.
Si los objetos de estado no tienen variables de instancia (es decir, el estado que representan está codificado completamente en su tipo), los contextos pueden compartir un objeto de estado.

Behavioral Patterns - State



Behavioral Patterns - State

```
const onState = { handle: () => console.log("ON") };
const offState = { handle: () => console.log("OFF") };

function createSwitch() {
  let state = offState;
  return {
    press: function () {
      state.handle();
      state = (state === offState) ? onState : offState;
    }
  };
}

const s = createSwitch();
s.press(); // OFF
s.press(); // ON
s.press(); // OFF
```



Behavioral Patterns - Strategy



Define una familia de algoritmos, encapsula cada uno y los hace intercambiables.

El patrón Strategy permite que el algoritmo varíe independientemente de los clientes que lo utilizan.

Behavioral Patterns - Strategy - Aplicación



Muchas clases relacionadas difieren únicamente en su comportamiento.

Las estrategias permiten configurar una clase con uno de varios comportamientos.

Una clase define muchos comportamientos, que aparecen como múltiples sentencias condicionales en sus operaciones.

En lugar de muchas sentencias condicionales, mueva las ramas condicionales relacionadas a su propia clase Strategy.

Behavioral Patterns - Strategy - Aplicación



Se necesitan diferentes variantes de un algoritmo.

Por ejemplo, se podrían definir algoritmos que reflejen diferentes equilibrios espacio-tiempo. Se pueden utilizar estrategias cuando estas variantes se implementan como una jerarquía de clases de algoritmos.

Un algoritmo utiliza datos que los clientes no deberían conocer.

Utilice el patrón Strategy para evitar exponer estructuras de datos complejas y específicas del algoritmo.

Behavioral Patterns - Strategy - Características



- Familias de algoritmos relacionados.

Las jerarquías de clases de estrategia definen una familia de algoritmos o comportamientos que los contextos pueden reutilizar. La herencia puede ayudar a identificar la funcionalidad común de los algoritmos.

- Una alternativa a la subclasificación.

Encapsular el algoritmo en clases Strategy independientes permite modificarlo independientemente de su contexto, lo que facilita su cambio, comprensión y extensión.

Behavioral Patterns - Strategy - Características



- Las estrategias eliminan las declaraciones condicionales.
El patrón Estrategia ofrece una alternativa a las declaraciones condicionales para seleccionar el comportamiento deseado.
- Selección de implementación.
Las estrategias pueden proporcionar diferentes implementaciones del mismo comportamiento. El cliente puede elegir entre estrategias con diferentes compensaciones de tiempo y espacio.

Behavioral Patterns - Strategy - Características



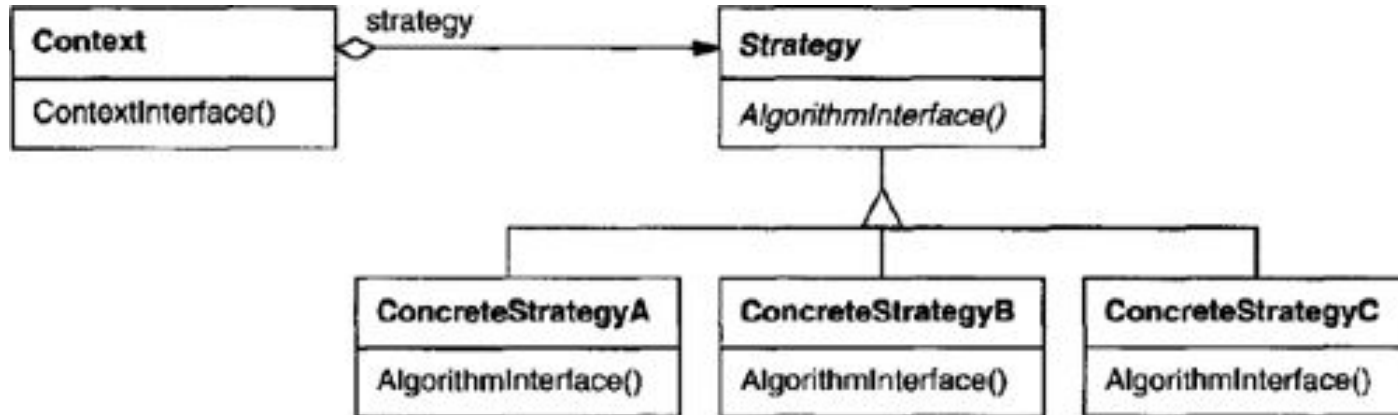
- Los clientes deben conocer las diferentes estrategias.

Este patrón presenta una posible desventaja: el cliente debe comprender cómo difieren las estrategias antes de poder seleccionar la adecuada.

- Mayor número de objetos.

Las estrategias aumentan el número de objetos en una aplicación. A veces, se puede reducir esta sobrecarga implementando estrategias como objetos sin estado que los contextos pueden compartir.

Behavioral Patterns - Strategy



Behavioral Patterns - Strategy

```
const add = { execute: arr => arr.reduce((a,b)=>a+b, 0) };  
const multiply = { execute: arr => arr.reduce((a,b)=>a*b, 1) };
```

```
function createContext(strategy) {  
  let strat = strategy;  
  return {  
    setStrategy: s => strat = s,  
    execute: arr => strat.execute(arr)  
  };  
}
```

```
const c = createContext(add);  
console.log(c.execute([1,2,3])); // 6  
c.setStrategy(multiply);  
console.log(c.execute([1,2,3])); // 6
```



Behavioral Patterns - Observer



Define una dependencia de uno a muchos objetos así cuando un objeto cambie de estado, todos sus dependientes sean notificados y actualizados automáticamente.

Behavioral Patterns - Observer - Aplicación



- Cuando una abstracción tiene dos enfoques, uno dependiente del otro, encapsularlos en objetos separados permite modificarlos y reutilizarlos de forma independiente.
- Cuando un cambio en un objeto requiere cambiar otros y no se sabe cuántos objetos deben cambiarse.

Behavioral Patterns - Observer - Aplicación



- Cuando un objeto debería poder notificar a otros objetos sin hacer suposiciones sobre quiénes son.
En otras palabras, no se desea que estos objetos estén estrechamente acoplados.

Behavioral Patterns - Observer - Características



- **Acoplamiento abstracto entre sujeto y observador.**
Un sujeto solo sabe que tiene una lista de observadores, cada uno conforme a la interfaz simple de la clase abstracta Observador. El sujeto desconoce la clase concreta de ningún observador. Por lo tanto, el acoplamiento entre sujetos y observadores es abstracto y mínimo.
- **Compatibilidad con Broadcasting.**
A diferencia de una solicitud normal, la notificación que envía un sujeto no necesita especificar su destinatario. La notificación se difunde automáticamente a todos los objetos interesados que se suscribieron.

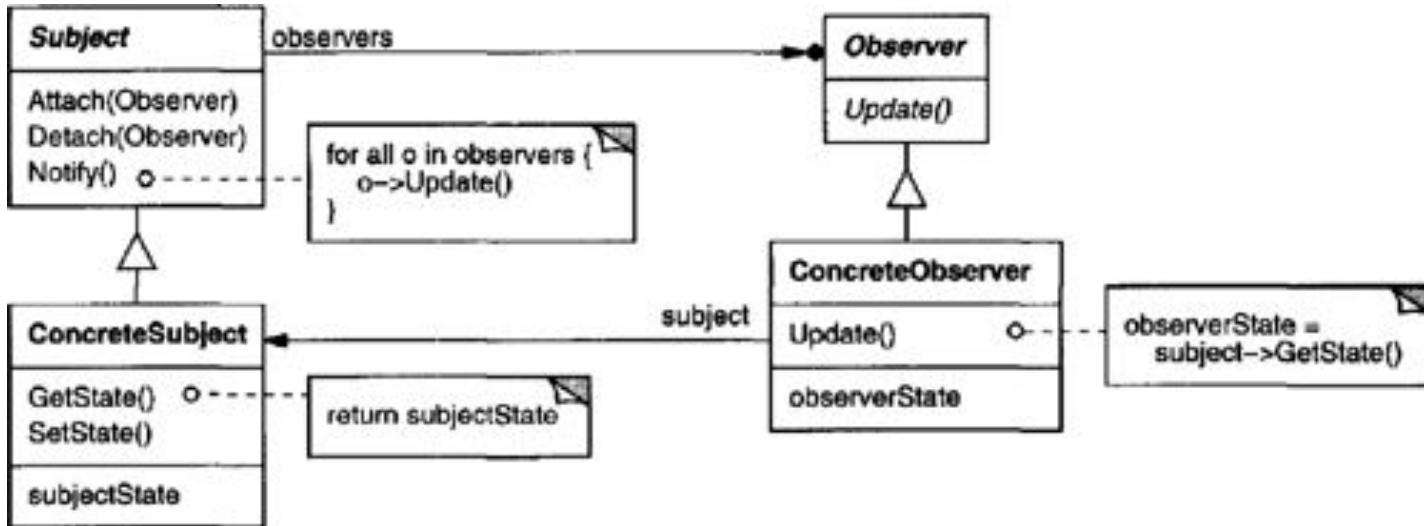
Behavioral Patterns - Observer - Características



- Actualizaciones inesperadas.

Dado que los observadores desconocen la presencia de los demás, pueden pasar por alto el coste final de cambiar de tema. Una operación aparentemente inocua sobre el tema puede provocar una cascada de actualizaciones para los observadores y sus objetos dependientes.

Behavioral Patterns - Observer



Behavioral Patterns - Observer

```
function createSubject() {  
  const observers = [];  
  return {  
    attach: obs => observers.push(obs),  
    notify: msg => observers.forEach(o => o.update(msg))  
  };  
}  
  
function createObserver() {  
  return { update: msg => console.log("Received:", msg) };  
}  
  
const s = createSubject();  
const o1 = createObserver(), o2 = createObserver();  
s.attach(o1);  
s.attach(o2);  
s.notify("Hello"); // Ambos observadores reciben "Hello"
```



Referencia



- Freeman E, Robson E (2020), Head First Design Patterns, 2nd Edition, O'Reilly Media, Inc.
- Gamma E, Helm R, Johnson R, Vlissides J (1994), Design Patterns: Elements of Reusable Object-Oriented Software, O'Reilly Media, Inc.

Próxima Clase



Module & Creational Pattern

JavaScript

